Amanda Murphy
Programming Assignment 4
Spell Checker Lab
5 November 2026

Description of the Problem: For this assignment, we were given two files: one file which was a list of approximately 130,000 words, and another file which was text from the book Oliver. The goal was to create a "spell checker" that would store words into an array of 26 Sequence objects, based on the letter of the alphabet the words started with. As an example, all words starting with the letter a would go into the first element, b would go into the second element, and so on. We then needed to develop a string parser that would read the book file, generate one word at a time, and then check that word against the dictionary sequence for the correct starting letter to see if the word was included in the collection. We needed to return how many words were found or not found. To demonstrate how efficient (or inefficient) this approach was, we also needed to track how many strings were traversed for words found and not found, and calculate the average.

Algorithms: In this lab, the array of 26 Sequence objects served as a basic Hash Table with separate chaining. The index was calculated by using the starting character which mapped to a bucket index from 0-25. To search for a word, the Sequence::search() function performed a linear search in O(N), meaning that the algorithm went through the Sequence until it found the word, or through the entire Sequence if it did not find the word. This is incredibly slow, and I was not expecting to see it take so long when I was running it on my computer, but I think this was a good demonstration of the inefficiency. Despite sorting the words into 26 Sequences, some letters are more common, which meant that every 's' word took much longer to search because there was a longer list of words for that Sequence.

Program Design: I implemented a linked list after watching the MIT video lecture, thinking this would be more efficient. So the **Node** class acted as a linked list element that contained a single word and a pointer to the next node. **Sequence** managed the singly linked list data structure and search logic. **WordParser** handled the I/O stream, character filtering, converting the characters to lowercase, and extracting the words. **Main.cpp** utilized all of the previous classes to create the dictionary array of Sequences, populate it, parse the text of Oliver, and output the results. I put a few helper methods in main.cpp, that I should probably migrate to one of the other classes.

One thing I did better in this lab is that I test each piece I wrote before going on to the next piece. I've overwhelmed myself in the past by writing too much at once.

Observations: In the past when I studied data structures and algorithms for interviews it was very much a hypothetical. It's hard to think of Big O notation in practice, but I thought it was really cool to see how slowly this solution ran. Originally I was thinking about sorting the words into sub-buckets for the first three letters of the word, so that you could have the word "hard" go into one bucket for h, a sub-bucket for a, and another within that sub-bucket for r, but after reading the instructions again, I realized that the point of this assignment was to demonstrate how slow this would run. I'm looking forward to future assignments where we can work to optimize this step by step.

```
Amandas-Mac-Studio:build helloamandamurphy$ ./assignment_4
Successfully loaded dictionary.
Words Found: 927260
Words Not Found: 72281
Total words: 999541
Average Comparisons Found: 3245
Average Comparisons Not Found: 7096
```